UQAC

Université du Québec
à Chicoutimi

Atelier 1 : Mobile Forensics

8INF968 - Atelier pratique en cybersécurité I

Fehmi Jaafar Samuel Desbiens

Justin Bossard Mattéo Gouhier Paul Mathé Samuel Plet

3 octobre 2025

Table des matières

Introduction **3**

1 *Identification* et *Collection* **4**

2 *Examination* **5**
2.1 Mise en place de l'environnement de travail . . . 5
2.1.1 Installation d'Autopsy . . . 5
2.1.2 Création du projet . . . 5
2.1.3 Ajout des données . . . 5
2.1.4 Exploitation des données . . . 5
2.2 Identité . . . 6
2.2.1 Profil Instagram . . . 6
2.2.2 Contacts . . . 7
2.3 Photographies . . . 7
2.3.1 Corrélation avec Instagram . . . 7
2.3.2 Localisation . . . 8
2.3.3 Transactions Bitcoin Cash (BCH) . . . 11
2.4 Messages . . . 12
2.4.1 Examen du code source d'OmniNotes . . . 12
2.4.2 Messages décodés . . . 15
2.5 Skimming . . . 15
2.5.1 Fonctionnement d'un *skimming device* . . . 17
2.5.2 Interprétation de notre audio . . . 18

3 *Analysis* **21**
3.1 Identité et lien avec HYDRA . . . 21
3.2 Voyage à Genève . . . 21
3.3 Implication dans le Skimming . . . 22

Conclusion **23**
Synthèse de l'enquête . . . 23
Appartenance à l'organisation HYDRA . . . 23
Implication dans les activités de skimming . . . 23
Géolocalisation et timeline . . . 23
Intégrité des preuves . . . 23
Recevabilité juridique . . . 23
Évaluation critique . . . 24
Apports pédagogiques . . . 24
Défis techniques rencontrés . . . 24

Sources **25**

Table des figures

2.1 Organisation des données dans Autopsy . . . 6
2.2 Informations civiles - `/system_ce/0/snapshots/141_reduced.jpg` . . . 6
2.3 Profil Instagram . . . 7
2.4 Photo de *piadina* sur Instagram . . . 8
2.5 Carte des photos géolocalisées . . . 8
2.6 Logo d'HYDRA - `/media/0/DCIM/Camera/20210408_182133.jpg` . . . 8
2.7 Gare de Bussigny sur Google Maps . . . 9
2.8 Hôtel à Route des Flumeaux 10, 1008 Prilly, Suisse . . . 9
2.9 Gare de Prilly sur Google Maps . . . 10
2.10 Gare de Renens sur Google Maps . . . 10
2.11 Vue du train - `/media/0/DCIM/Camera/20210415_091733.jpg` . . . 10
2.12 ICC - `/media/0/DCIM/Camera/20210415_100517.jpg` . . . 10
2.13 Billet d'avion - `/clipboard/2139570144642_471_195/Screenshot_20210418-174639_Chrome.jpg` . . . 11
2.14 Adresse BCH - `/clipboard/1740051371957_356_87/Screenshot_20210418-161954_BRD.jpg_thum.jpg` . . . 11
2.15 Spectre de l'audio (Audacity) . . . 16
2.16 Skimming device . . . 16
2.17 Exemple d'audioskim . . . 17
2.18 Polarisation d'une carte magnétique . . . 17
2.19 Fonctionnement d'un skimming device . . . 18
2.20 Chaîne de traitement d'un ATM . . . 18
2.21 Horloge et décodage F2F . . . 19
2.22 F2F . . . 19
2.23 Magstripper . . . 19
2.24 Credit Card validator . . . 20

3.1 Localisation du Consulat de Turquie à Genève . . . 21

Introduction

L'objectif de ce premier projet de Mobile Forensic est de réaliser une investigation sur une image de smartphone, dans un contexte préétabli, afin de mettre en évidence des preuves numériques pertinentes pour une enquête.

Par définition, la *Digital Forensics*, ici appliquée aux environnements mobiles, regroupe *l'ensemble de méthodes et d'outils visant à récupérer, préserver et examiner des preuves numériques issues ou transmises par des dispositifs électroniques.*
Ce domaine s'appuie sur le principe de Locard[1], et vise à répondre aux 5WH[2].

L'investigation doit préserver l'intégrité des preuves et garantir une acquisition recevable juridiquement, en maintenant une chaîne de confiance.

Le processus légal d'investigation est souvent le suivant :

1. **Identification** Détection de l'incident, reconnaissance et sécurisation des sources de preuves.
2. **Collection** Acquisition des données de l'appareil avec des méthodes garantissant l'intégrité (copies, empreintes numériques).
3. **Examination** Extraction, tri et préparation des données pertinentes (fichiers, métadonnées, artefacts cachés).
4. **Analysis** Interprétation des preuves pour établir les faits, reconstituer les événements et relier les acteurs.
5. **Presentation** Restitution structurée des résultats sous forme de rapport recevable juridiquement.

Pour ce travail, les étapes d'*Identification* et de *Collection* ont déjà été effectuées. Nous les présenterons dans un premier temps, avant de présenter successivement nos résultats pour *Examination* et *Analysis*. Ce rapport, dans son ensemble, constitue la phase de *Presentation*.

[1] Chaque interaction laisse une trace
[2] Who, what, when, where, why, and how

Partie 1

Identification et *Collection*

> L'enquête présentée dans ce rapport s'inscrit dans une progression en 5 étapes du Challenge 2021 de la DFRWS, nous en sommes à la troisième étape.

Le 9 avril 2021, un dispositif de skimming a été découvert sur un distributeur automatique de la Poste suisse, à Lausanne. Une micro-SD intégrée au dispositif a été saisie et a révélé des traces d'utilisation frauduleuse liées à des cartes bancaires.

À partir de ces informations, une perquisition a été menée le 18 avril 2021 à Aoste, Italie, où un laboratoire de fabrication a été découvert. Un Raspberry Pi connecté à une imprimante 3D a pu être saisi. Une copie de sa micro-SD a permis d'étudier son rôle éventuel dans la production d'objets illicites.

L'exploitation de ces premiers résultats a conduit à l'identification d'un suspect, interpellé le 20 avril 2021 à l'aéroport de Genève alors qu'il tentait de voyager avec un billet acheté à l'aide d'une carte bancaire volée. Son téléphone a été saisi et une image complète du système de fichiers a été extraite le lendemain. Une analyse préliminaire a révélé la présence de SMS encodés, susceptibles de contenir des éléments de preuve pertinents pour la compréhension des faits.

Notre investigation se basera donc sur l'image en question.

- https://cfreds.nist.gov/all/FrancescoServida%2FDFRWS/SamsungGalaxyS10Android10DFRWS2021
- SHA2-256 : `54877505f1b4eb26c4cb6b43fd6338424660c207e678b773044a4a79d6e374b7`

Partie 2

Examination

La racine des chemins correspond à `Dump/data/`

Pour examiner l'image, nous allons utiliser principalement le logiciel Autopsy. Celui-ci permet d'analyser et d'extraire toutes les informations potentiellement utiles d'une image disque : métadonnées, conversations, journal d'appels, mails, contacts…

Nous allons commencer par montrer le plus visible, en allant vers les données plus compliquées à extraire, ce qui inclut les messages encodés, mentionnés précédemment comme étant pertinents.

2.1 Mise en place de l'environnement de travail

2.1.1 Installation d'Autopsy

Pour installer Autopsy, il suffit de se rendre sur la page de téléchargement. On peut l'installer sur GNU/Linux, MacOS et Windows, les instructions pour chaque OS sont détaillées sur le site officiel.

2.1.2 Création du projet

Après installation, on peut créer le projet. On démarre Autopsy, on sélectionne `New Case`, puis on nomme le projet `Samsung`, par exemple.
On remplit les informations optionnelles de manière adéquate, puis on finit la création du projet.

2.1.3 Ajout des données

Après création du projet, on peut ajouter les données. Dans l'ordre :

1. `Generate new host name based on data source name`
2. `Logical files`
3. `Local files and folders`, puis on ajoute le `zip` directement.
4. On laisse tout coché, sauf `iOS analyzer` (et `Android Analyzer (aLEAPP)` si on n'a pas Windows)
5. On termine, et on attend patiemment la fin de l'analyse

2.1.4 Exploitation des données

On a, à la fin de l'analyse, toutes les données retrouvées organisées dans différentes catégories.

Cela va nous permettre d'examiner facilement toutes les données d'intérêt, chose que nous allons maintenant présenter.

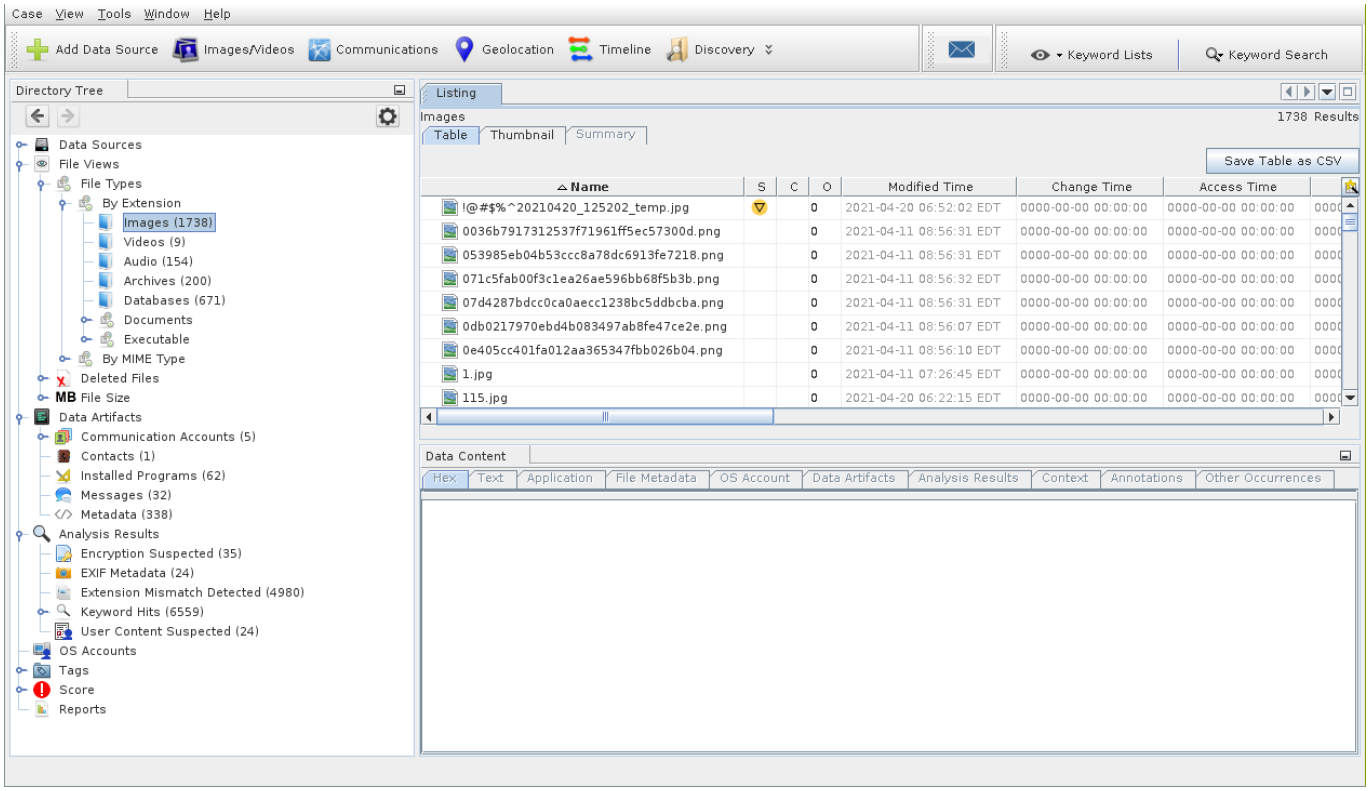


Figure 2.1: Organisation des données dans Autopsy

2.2 Identité

Commençons par présenter le propriétaire du téléphone : il se nomme Werner von Strucker.

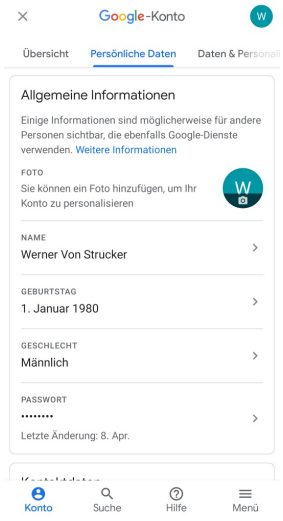


Figure 2.2: Informations civiles - `/system_ce/0/snapshots/141_reduced.jpg`

2.2.1 Profil Instagram

L'analyse des fichiers d'application a révélé la présence d'Instagram sur l'appareil. L'examen du fichier de préférences `/data/data/com.instagram.android/shared_prefs/com.instagram.android_preferences.xml` a permis d'identifier le profil utilisateur :

```
username\&quot;:\&quot;wernervonstrucker\&quot;,\&quot;full_name\&quot;:\&quot;
↪ Werner Von Strucker
```

Cette découverte confirme l'identité du propriétaire et révèle l'utilisation du nom d'utilisateur `wernervonstrucker` sur la plateforme Instagram.

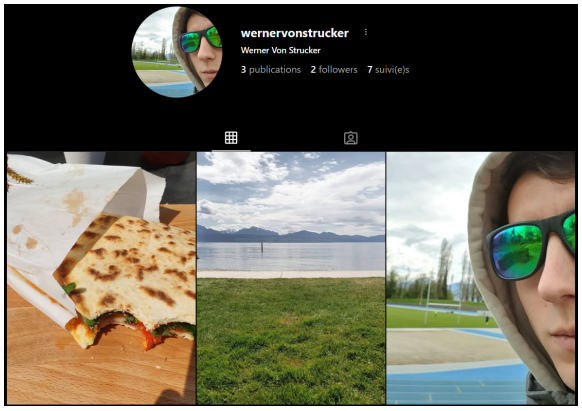


Figure 2.3: Profil Instagram

2.2.2 Contacts

Une base de données GMail[1] donne un aperçu de ses contacts par mail. Notons les adresses qui ne proviennent pas de marques :

- `vonstruckerwerner@gmail.com` : vraisemblablement son adresse personnelle.
- `therealredskull41@gmail.com` : nous reviendrons sur Red Skull dans l'*Analysis*, voir 3.
- `arthur.kart@yopmail.com` : un contact personnel. Nous n'avons rien trouvé de plus à son sujet.

2.3 Photographies

Dans les photos, un certain nombre semble intéressant.

2.3.1 Corrélation avec Instagram

Deux des images postées sur Instagram correspondent exactement aux photos trouvées sur l'appareil mobile :

1. **Le selfie** : Identique à `/media/0/DCIM/Camera/IMG_20210411_144226.jpg` trouvé dans la galerie du téléphone
2. **La photo de piadina** : Correspondant à `/media/0/DCIM/20210420_125202.jpg`, également trouvée sur l'appareil

   On remarque également un lieu posté avec ce met : **Piadineria Mamma Mia**, un restaurant italien situé à Lausanne.

Cette corrélation confirme de manière irréfutable l'appartenance du profil Instagram au propriétaire de l'appareil et établit un lien direct entre l'activité en ligne et les données présentes sur le dispositif mobile.

[1] `/data/com.google.android.gm/databases/peopleCache_vonstruckerwerner@gmail.com_com.google_11.db`

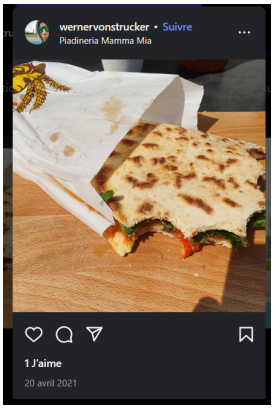


Figure 2.4: Photo de *piadina* sur Instagram

2.3.2 Localisation

16 photos ont une localisation associée. Autopsy permet d'extraire toutes ces localisations, et de les représenter sur une carte.
On voit donc que l'utilisateur en a pris uniquement en Suisse, principalement aux abords de Lausanne.



Figure 2.5: Carte des photos géolocalisées

Cette observation est corroborée par le restaurant mentionné sur Instagram et confirme la présence du suspect dans la région lausannoise, renforçant ainsi la cohérence géographique de ses déplacements.

Parmi les photos intéressantes, on remarque une photo du logo d'HYDRA, datant du 8 avril 2021, avec un lien. Celui-ci est inaccessible, et nous n'avons pas pu trouvé d'informations à son sujet. Elle a été prise à l'Université de Lausanne, selon sa localisation.

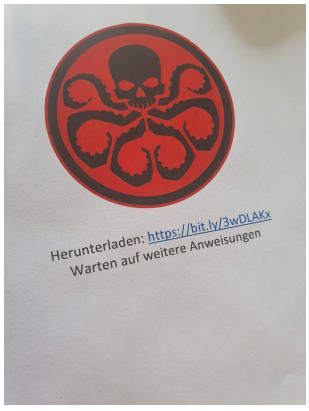


Figure 2.6: Logo d'HYDRA - `/media/0/DCIM/Camera/20210408_182133.jpg`

Il y a deux autres photos à l'Université, cela montre qu'il est potentiellement professeur ou chercheur, et peut-être étudiant[2].

On trouve par ailleurs le fichier `/data/com.google.android.apps.maps/cache/image_manager_disk_cache/08c1ccbdafb163c39925d5bbd6bab77c9f4c6012590ec3552e109f6026e591b3.0`, qui est une image cache de Google Maps de la commune de Bussigny, attenante à Lausanne, plus précisément de sa gare.



Figure 2.7: Gare de Bussigny sur Google Maps

On a aussi un hôtel[3] (en retrouvant l'adresse indiquée par le pointeur) et la gare[4] de Prilly, une commune proche de Lausanne aussi.

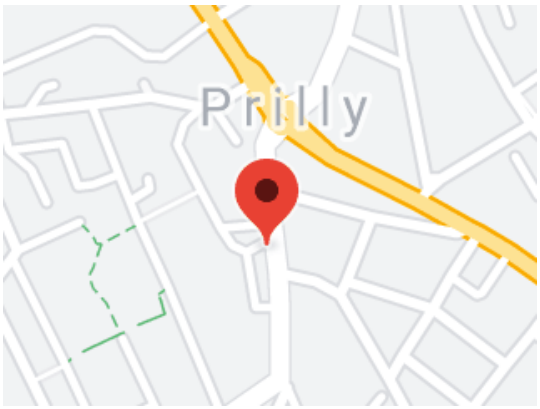


Figure 2.8: Hôtel à Route des Flumeaux 10, 1008 Prilly, Suisse

Enfin, on trouve la gare de Renens[5], une troisième commune attenante à Lausanne.

De plus, un évènement de calendrier dans `/data/com.android.providers.calendar/databases/calendar.db-journal` renseigne un train entre Lausanne et Genève prévu pour le 20 avril à 16 h 45.

On remarque aussi deux photos éloignées de Lausanne, une prise en train, le 15 avril 2021 à 9 h 17, et une devant ICC[6], à Genève, le même jour, à 10 h 05.

Enfin, une capture d'écran d'un vol Genève - New York, pour le 20 avril 2021, est retrouvé dans le cache du presse-papier. Il s'agit du vol avant lequel Werner a été appréhendé.

[2] Puisqu'il serait né le 01/01/1980 d'après la Figure 2.2

[3] `/data/com.google.android.apps.maps/cache/image_manager_disk_cache/1d5071cb0596e4eed60b8733663f2c720fa7d8c7f0d7645a2f5da89030c71c45.0`

[4] `/data/com.google.android.apps.maps/cache/image_manager_disk_cache/b917b3ad6904cf85ccbabe0c1d5103862540ebc5be61a2644a19568a102a3484.0`

[5] `/data/com.google.android.apps.maps/cache/image_manager_disk_cache/c231692d67d3c61df5f9aabdc84de244b22bfa0453ec5c3d6de5d56daeca21ab.0`

[6] ICC est un centre d'affaires, proche de l'aéroport de Genève, qui loue des bureaux.



Figure 2.9: Gare de Prilly sur Google Maps



Figure 2.10: Gare de Renens sur Google Maps



Figure 2.11: Vue du train - `/media/0/DCIM/Camera/20210415_091733.jpg`



Figure 2.12: ICC - `/media/0/DCIM/Camera/20210415_100517.jpg`

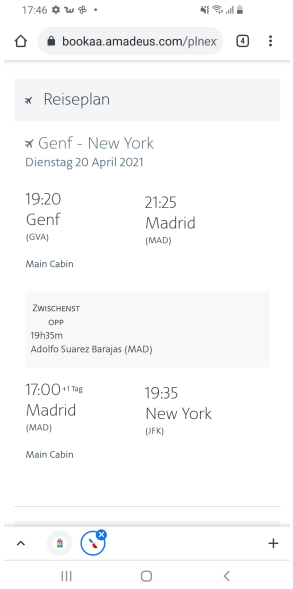


Figure 2.13: Billet d'avion - `/clipboard/2139570144642_471_195/Screenshot_20210418-174639_Chrome.jpg`

Pour mieux comprendre la situation, il est nécessaire de déchiffrer les messages. Auparavant, terminons l'analyse des photos.

2.3.3 Transactions Bitcoin Cash (BCH)

> On identifie les adresses BCH avec leurs 4 premiers et leurs 4 derniers caractères. On note une transaction de la même manière.

Une capture d'écran avec l'adresse BCH qpuv-d23y est présente.



Figure 2.14: Adresse BCH - `/clipboard/1740051371957_356_87/Screenshot_20210418-161954_BRD.jpg_thum.jpg`

En regardant sur la blockchain, on retrouve l'historique de transactions de cette adresse :

- d14c-d8f2
  - Notre adresse a envoyé, avec qzef-sezu, 0.00730765 BCH ($4.34) à qp9d-qxyj.

- L'adresse de destination n'a aucune autre transaction, mais l'autre adresse d'envoi a 9084-0ac7
    - qzkl-kv4n et qzs9-4k0c ont envoyé 0.00680453 BCH ($4,04) à qrxs-8h5p et qzef-sezu
    - Les autres adresses ont d'autres transactions et adresses associés

- bddf-e925
    - qqvl-dnqz a envoyé 0.00175909 BCH ($1,04) à qpuh-8g3g et notre adresse
    - L'adresse d'envoi a aussi la transaction 2845-de5f associée
        - ...

Et ainsi de suite. Toutes les transactions semblent avoir eu lieu le 18 avril 2021.
On a donc trouvé de nombreuses adresses BCH, dont les transactions s'entremêlent, pour ce qui semble être de l'obfuscation. Cela pourrait permettre de masquer ou compliquer :

- la chaîne de provenance (quelle adresse/quel portefeuille a fourni les fonds)
- la chaîne de destination (où les fonds vont ensuite)
- l'identité/attribution (lien entre adresse et personne ou service)
- la chronologie (quand les transferts ont eu lieu, y compris ordre et corrélation temporelle)
- le montant réel (par fragmentation ou autre)

2.4 Messages

L'examen des données extraites du téléphone du suspect a permis de révéler la présence de messages illisibles reçus et envoyés, suggérant l'utilisation de méthodes d'encryption des messages.

Par exemple : `2 2 +41786909109 1617901761280 0 1 -1 2 HHY6cKEMFlNU3NPs/bxhxPKQRyDXUrQK7a8tHtX5xBXdYQ= 0 0 1 it.feio.android.omninotes 1 0 0 228015930119996 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0`

L'analyse des métadonnées associées à ces messages met en évidence la provenance de ces messages, OmniNotes, une application open-source de prise de notes. Voir des messages cryptés depuis une application de prise de notes a priori non dédiée à l'envoi de message est un indicateur fort de tentative de dissimulation.

La prochaine étape est d'analyser le code source de cette application présent sur l'appareil pour identifier les méthodes utilisées.

2.4.1 Examen du code source d'OmniNotes

Pour extraire le code de l'application présente sur l'appareil, nous avons utilisé apk2java-linux.

L'examen des fonctions a confirmé son rôle dans l'envoi de messages, notamment via l'appel d'une fonction dédiée à l'envoi.

```
1 public void putComm(View object) {
2     object = (EditText)this.findViewById(2131296976);
3         object = new
  ↪ TextHelper(this.getResources()).put(object.getText().toString());
4         if (object.length() > 160) {
5             Toast.makeText((Context)this,
  ↪ (CharSequence)ZipHelper.decomp("eJwFQLENwDAIe8UH5I1u6ZL2gQgs1AUPqEuuR1eanI
  ↪ 6bVTuIV8JUxsCi/0Y832ED99UM+Q=="), (int)0).show();
6             return;
```

```
        }
        if (object.equals((Object)"")) {
            Toast.makeText((Context)this,
↪   (CharSequence)ZipHelper.decomp("eJwFgLEJACAMBFfJMpaCK6R4QhqV+/mLsOARB/2kb8
↪   WWnaUBZ9oIuA=="), (int)0).show();
            return;
        }
        StringBuilder stringBuilder = new StringBuilder();
        stringBuilder.append(ZipHelper.decomp("eJwFQLEJADAMesU7+kOmXlCoSBYX7x
↪   /kOn8tDJMnHhQ48AYI"));
        stringBuilder.append((String)object);
        stringBuilder.append(" to ");
        stringBuilder.append(this.rsn());
        Toast.makeText((Context)this, (CharSequence)stringBuilder.toString(),
↪   (int)0).show();
        PendingIntent.getActivity((Context)this, (int)0, (Intent)new
↪   Intent((Context)this, CommActivity.class), (int)0);
        SmsManager.getDefault().sendTextMessage(this.rsn(), null, (String)object,
↪   null, null);
}
```

L'analyse a mis en évidence l'utilisation de blocs de données encodés :

```
private String get_data() throws IOException, JSONException {
        String string2 =
         ↪  ZipHelper.decomp("eJwNyUsKwDAIBNATZSTbHietkH5wShUkt7frNyNe30Tm6hg
         ↪  PB3Z+ijztYDpM45eW3rrcunA5rQDMDhIV");
        Object object = new StringBuilder();
        object.append(string2);
        object.append(ZipHelper.decomp("eJwFwIEIAAAAACC0/a1OAEAAQA=="));
        object.append(ZipHelper.decomp(this.resources.getString(2131821101)));
        string2 = (HttpsURLConnection)new URL(object.toString()).openConnection();
        try {
            String string3;
            object = new BufferedReader((Reader)new
↪   InputStreamReader(string2.getInputStream()));
            Object object2 = new StringBuffer();
            while ((string3 = object.readLine()) != null) {
                object2.append(string3);
            }
            object = new
↪   JSONObject(String.valueOf((Object)object2)).getString("key");
            object2 = new
↪   SimpleDateFormat(ZipHelper.decomp("eJwNwIEAAAAMhECiULJ4idm3u4eyBRWSA6E="),
↪   Locale.getDefault()).format(new Date());
            string3 = new StringBuilder();
            string3.append((String)object);
            string3.append((String)object2);
            object = string3.toString();
            return object;
        }
        finally {
            string2.disconnect();
```

```
        }
}
```

Avec des bouts de code codés en `base64` et compressé en zlib :

- `eJwNyUsKwDAIBNATZSTbHietkH5wShUkt7frNyNe30Tm6hgPB3Z+ijztYDpM45eW3rrcunA5rQ DMDhIV` nous donne le lien vers un site nous permettant de récupérer une clé : `rb5CuefkDLyb9T`
- `eJwNwIEAAAAMhECiULJ4idm3u4eyBRWSA6E` nous donne `yyyy-MM-dd`.

En analysant ce bout de code on comprend que l'on va récupérer la clé sur le site Web, on récupère aussi la date actuelle que l'on va mettre sous format `yyyy-MM-dd` et que l'on va concaténer à la clé. Ce qui va nous donner une nouvelle clé permettant de coder les messages à partir d'une clé statique et d'une clé dépendant de paramètres temporels.
Par exemple, un message envoyé le 15 Avril 2021 sera codé via la clé `rb5CuefkDLyb9T2021-04-15`.

Le code a permis de déterminer l'utilisation de l'algorithme de chiffrement par bloc AES dans le mode ECB avec le schéma PKCS5Padding :

```
private byte[] save_msg(String string2) throws NoSuchAlgorithmException,
 ↪  NoSuchPaddingException, InvalidKeyException, InvalidParameterSpecException,
 ↪  IllegalBlockSizeException, BadPaddingException, IOException, JSONException {
        SecretKeySpec secretKeySpec = new
         ↪  SecretKeySpec(this.get_data().getBytes(), "AES");
        Cipher cipher = Cipher.getInstance((String)"AES/ECB/PKCS5Padding");
        cipher.init(1, (Key)secretKeySpec);
        return cipher.doFinal(string2.getBytes("UTF-8"));
}
```

Après avoir crypté le message de cette façon, on va rajouter `HHY` devant ce message pour que l'application puisse différencier les messages chiffrés de ceux qui ne le sont pas.

```
public String put(String string2) {
        try {
            StringBuilder stringBuilder = new StringBuilder();
            stringBuilder.append("HHY");

 ↪  stringBuilder.append(Base64.encodeToString((byte[])this.save_msg(string2),
 ↪  (int)2));
            return stringBuilder.toString();
        }
        catch (IOException | InvalidKeyException | NoSuchAlgorithmException |
         ↪  InvalidParameterSpecException | BadPaddingException |
         ↪  IllegalBlockSizeException | NoSuchPaddingException | JSONException
         ↪  noSuchPaddingException) {
            return "";
        }
}
```

Pour résumer, on va coder les messages de la manière suivante :

$$\text{HHY} + \text{base64}(\text{AES avec clé hybride}(\text{Message Clair}))$$

Pour décoder on va donc :

1. enlever `HHY` du message crypté

2. traduire depuis la base64
3. décoder AES grâce à la clé créée à partir de `rb5CuefkDLyb9T` et de la date d'envoi du message

Cela nous a permis de pouvoir lire les 42 messages qui ont été jusqu'alors illisibles.

2.4.2 Messages décodés

Voici une partie de ces messages considérés comme les plus importants pour notre travail.

2.4.2.1 8 avril 2021

Werner : Heil Hydra 🐙!
6cKEMFlNU3NPs/bxhxPKQRyDXUrQK7a8tHtX5xBXdYQ=

2.4.2.2 15 avril 2021

Entrant : We need to meet you. Get to geneva asap, further instructions will follow.
HHYbm0jyH9MsbIM1DsyvNU9iDm63RQa9/1Fxfrs/A75m3wAM04VwDGphJ4m9IUT-jPQ//3qbJmQkxQ Qeu50WmK0zLHj93BiByuFEHsvHwzcfHsI
Werner : understood, was headed to geneva anyway, already on a train.
S6+p9IajgCnkXIm0tou uBzw/c5dSk4ZVzCEeA7BZmnR7+J5ybgmhLeXCyxpwovM8yFaK1NQt-fAJXHKwHnMGIxg==
Entrant : Good. Meet us at our base. Below the Turkish consulate. Delete this message once read!
HHYcdZD5nSpEbiNiPHaMn/eJR3qfnG1bBjxHLgN2Q91Jjx+DaAtq15BuatyPqSSH0ZOqLP-bis2 11525r4RIkumQSF1p8xjb7tbOHzWN250Sqmoh9v+9kcE6lnWMOcYHo4ub
Werner : Installed device at airport as discussed
HHY6JGyTGtl0IjI8PgtU2T/w5YPNCw1PhZ/Lagr4MhnCQhX+q36vX3TeUBZ6r/odTJU

2.4.2.3 18 avril 2021

Entrant: We sent you the payment to the address you provided us. confirm reception.
HHYqDAJ2HSbMahjfWcjFcL8H41ALcW17JkzdXUV02H4eHGTt+6mU+twizvAsvdMU6FkIUHhrP-cLzbiBsCV99j/skylD1A6fR7cocoqjOb61DM8=

Entrant : Btw Lay low, it seems your little fuckup with the Postomat[7] had big consequences.
HHY97NA5dDGdqomydjZt0GGycKgRS2T2vltp40yuOAo9azlfHzaNR3r2Fw78GXBuVC4YIif-Clzf+VKxxK13O2MgHwKDeGxIGva+uFj7wL+YnpKhdeGdwLiZ2/wszZN9dKlx
Werner : will do. and again my apologies, will never happen again, i've installed the others much more carefully.
HHYgSQ2GGQd5ZEvKdOk5qRrWfoEH0baqjRkTwacpwgxp02qqrNOueQ0z31KEiL65EZCnHr-jYy+7qbqFiVcCJCBnXnQdMC1MI64+dyXkuK6e/9I/ebQM3ZaAhqNLbIHNPOkTv35Hz1VseD-qLE1YpOPI4YQ==

2.5 Skimming

L'enregistrement intitulé `2021_04_09T1621` présente des éléments suspects. Après une première écoute, il apparaît presque vide, ne contenant que deux bruits sourds.

Or, la personne mise en cause dans cette affaire fait l'objet de soupçons de fraude à la carte bancaire et les autorités ont retrouvés un dispositif suspect sur un distributeur automatique de billets (ATM) à Lausanne, comme expliqué dans la partie identification de ce rapport.

[7] Nom des distributeurs de billets en Suisse

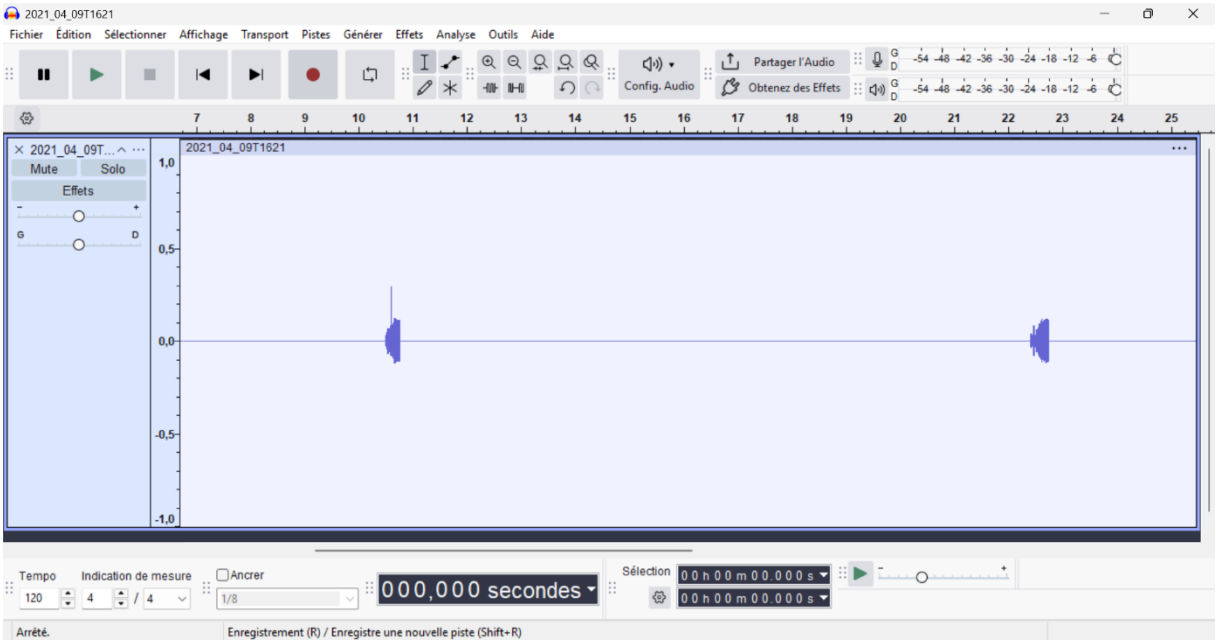


Figure 2.15: Spectre de l'audio (Audacity)

Après quelques recherches, il apparaît que l'ATM pourrait correspondre à un skimming device, c'est-à-dire un dispositif destiné à copier frauduleusement les informations contenues sur la bande magnétique d'une carte bancaire lorsque celle-ci est insérée dans un lecteur.

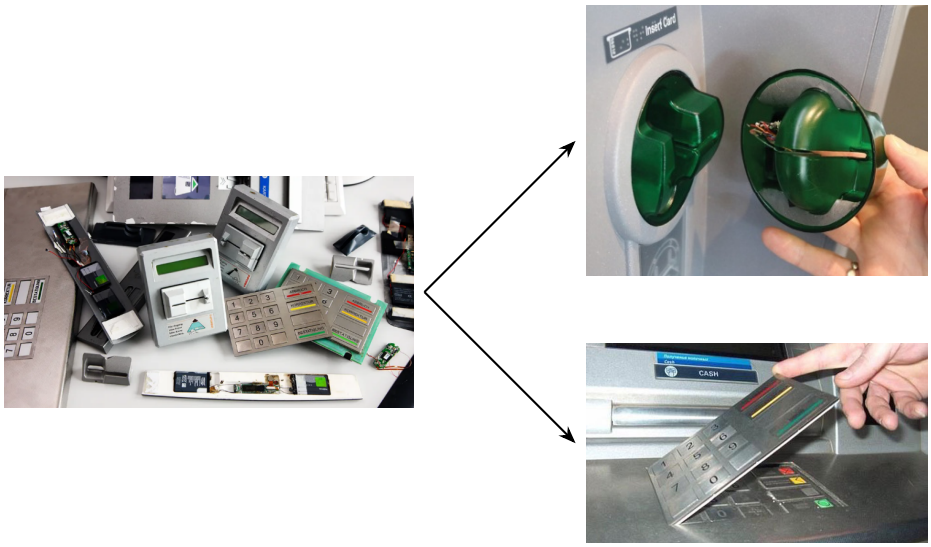


Figure 2.16: Skimming device

Des exemples de skimming devices disponibles en ligne montrent que certains comportent également une piste audio et sont appelés « audioskim ». Il est donc plausible que l'enregistrement provienne de cet appareil. De plus, le titre de l'enregistrement, incluant une date (`2021_04_09T1621`), suggère qu'il correspond à une capture audio réalisée à un instant précis et conforte la thèse de l'enregistrement de données de carte bancaire.

Pour arriver à décoder la piste audio, il faut comprendre la manière dont fonctionne les skimming devices. On peut voir leur architecture comme une version miniature d'un lecteur de bande magnétique, communément utilisé dans les systèmes d'accès par carte.

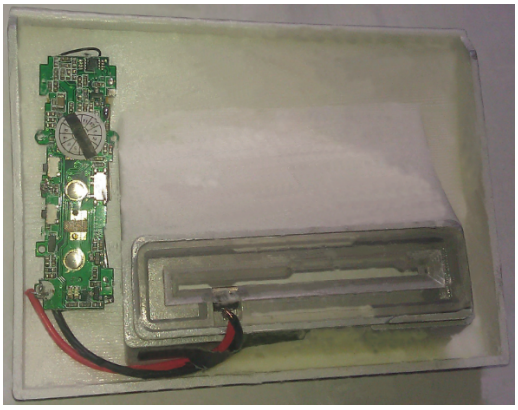


Figure 2.17: Exemple d'audioskim

2.5.1 Fonctionnement d'un *skimming device*

Tout d'abord, les cartes magnétiques sont composées de 3 pistes sur leur bande magnétique. On les appelle les track (en anglais) et chacune est composée de petites particules aimantées qui peuvent être polarisées dans deux directions : nord ou sud. Chaque polarisation correspond à un 1 ou un 0 en langage binaire.



Figure 2.18: Polarisation d'une carte magnétique

Table 2.1: Présentation des trois tracks

Track	Bits / caractère	Caractères possibles	Caractères max	Usage typique	Structure standard
1	7 bits (6 data + 1 parité)	Lettres majuscules, chiffres, ponctuation	**79**	Nom du porteur + PAN	`%B[ PAN ]^[ NOM ]^[ ... ]?`
2	5 bits (4 data + 1 parité)	Chiffres (0–9) + ;, =, ?	**40**	PAN (numéro de carte), date exp., code service	`;[ PAN ]=[ données additionnelles ]?`
3	5 bits (4 data + 1 parité)	Chiffres et symboles (similaire Track-2)	**107**	Rare (applications locales, transport, bancaire)	Format non standardisé (varie)

Le tableau ci-dessus présente les trois pistes (tracks) des cartes magnétiques et leurs caractéristiques d'encodage, en précisant le type d'informations stockées et leur capacité.

La track 2, utilisée principalement pour les cartes bancaires, stocke les informations essentielles aux transactions, telles que le numéro de compte, la date d'expiration et d'autres données nécessaires à l'authentification et au traitement sécurisé des paiements.

Quand on passe la carte dans un lecteur, celui-ci détecte les variations de polarisation de la bande magnétique et les traduit en informations numériques, ce qui permet d'identifier la carte et d'accéder aux données stockées. À l'intérieur du lecteur, une véritable chaîne de traitement prend en charge ce processus : une bobine capte les changements de polarité et un microcontrôleur analyse ces signaux pour reconstruire les données codées sur la carte, comme illustré sur le schéma 2.19.

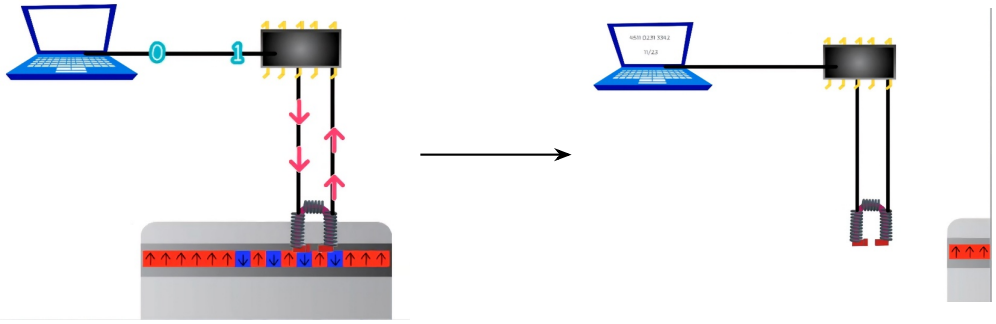


Figure 2.19: Fonctionnement d'un skimming device

2.5.2 Interprétation de notre audio

Voici un résumé détaillé, étape par étape, de la chaîne de traitement, illustrant comment le lecteur capte, analyse et interprète les données stockées sur la carte magnétique.

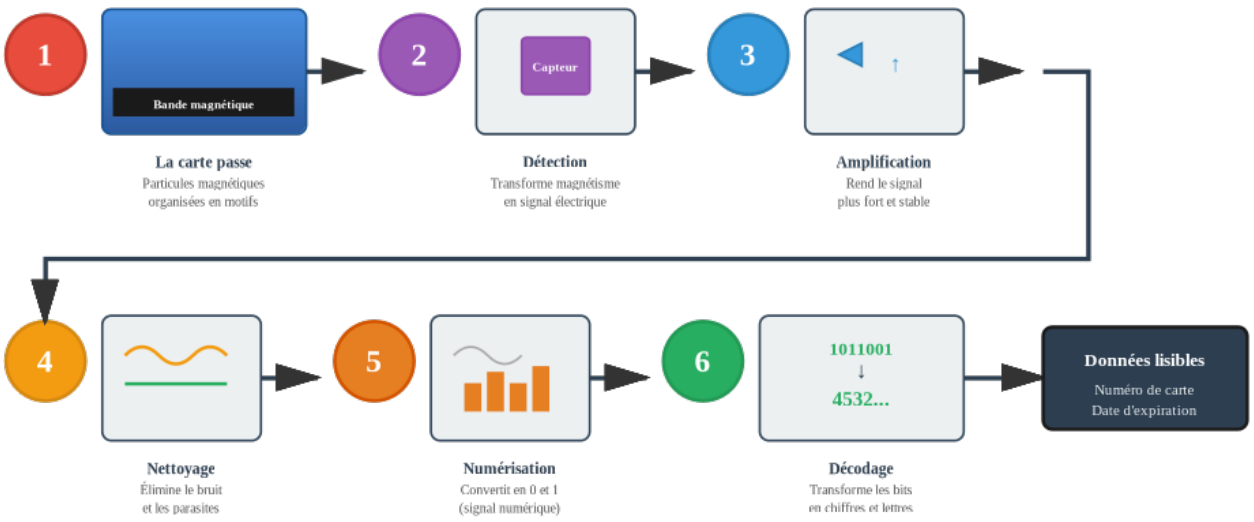


Figure 2.20: Chaîne de traitement d'un ATM

Dans notre cas, on se trouve à la fin de l'étape 4, il faut désormais convertir la piste audio en bits puis décoder ceci suivant un procédé de décodage propre aux track 2. En résumé, nous devons réaliser ces 2 étapes : réglage horloge et décodage F2F[8].

[8]Frequency To Frequency



Figure 2.21: Horloge et décodage F2F

Le codage F2F représente les bits par des changements de polarité sur la bande magnétique. Pour décoder, on regarde les transitions : une transition au début correspond à un 0, et des transitions au début et au milieu correspondent à un 1.

Le lecteur ou le programme analyse ces transitions pour reconstruire les 0 et 1 et extraire les données.

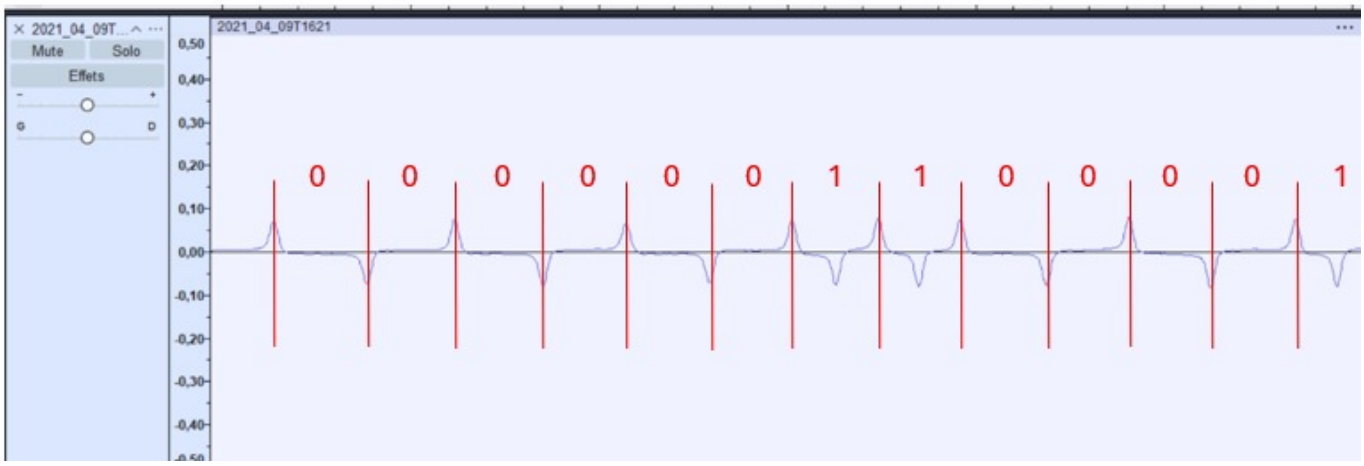


Figure 2.22: F2F

Ici, pour convertir le signal en bits on utilise Magstripper, un logiciel qui permet de récupérer le binaire à partir d'un fichier `.wav` (il réalise les opérations de récupération des horloges et de conversion des transitions en bits).

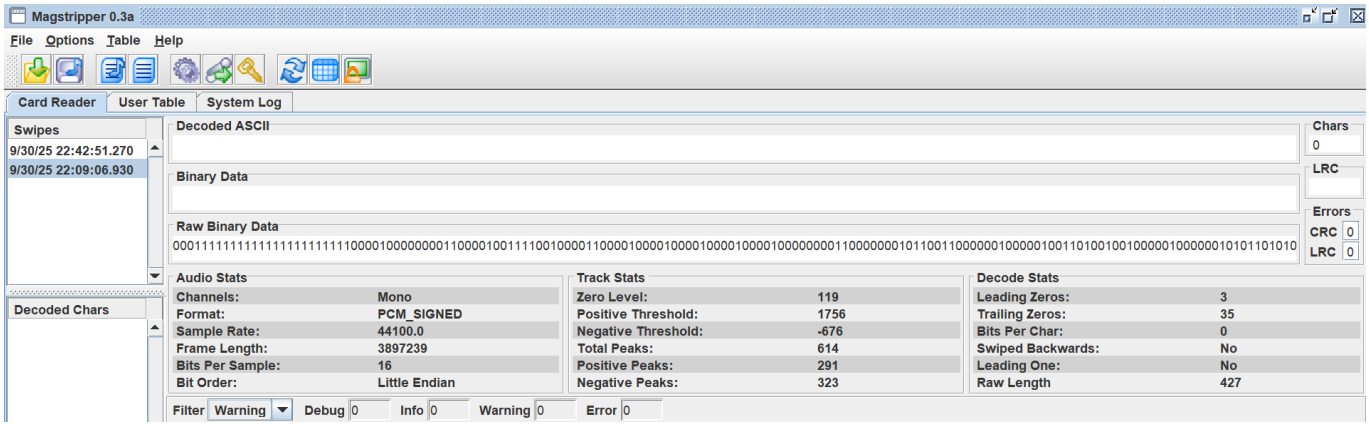


Figure 2.23: Magstripper

Pour des raisons légales, aucun outil en ligne n'autorise le décodage direct du format binaire de la piste 2 d'une carte bancaire ; c'est pourquoi nous avons développé un code python dédié.

```python
bits = "000111111111111111111111110000100000000110000100111110010000011..."

table = "0123456789:;<=>?"

```

```
for i in range(0, len(bits), 5):
    groupe = bits[i:i+5]
    if len(groupe) == 5:
        print(table[int(groupe[:4], 2)], end='')

print()
```

Ce code décode des données binaires de carte magnétique en parcourant une longue chaîne de 0 et de 1 par groupes de 5 bits à la fois grâce à la boucle `for i in range(0, len(bits), 5)` qui saute de 5 en 5 caractères (positions 0, 5, 10, 15, etc.).

Pour chaque groupe de 5 bits extrait avec `bits[i:i+5]`, le code vérifie qu'il contient bien 5 caractères, puis prend uniquement les 4 premiers bits avec `groupe[:4]` (le 5ème étant un bit de parité utilisé pour détecter les erreurs), convertit ces 4 bits de binaire en nombre décimal avec `int(..., 2)` (par exemple "0001" devient 1, "1111" devient 15), et utilise ce nombre comme index pour récupérer le caractère correspondant dans la table de conversion "0123456789:;<=>?", où chaque position représente un caractère valide du format Track 2 des cartes magnétiques (les chiffres 0-9, les séparateurs ; et =, et le terminateur ?).

Le `print(..., end='')` affiche chaque caractère décodé sans retour à la ligne pour former une chaîne continue, et le `print()` final ajoute un retour à la ligne à la fin, produisant ainsi le contenu lisible de la piste magnétique qui contient typiquement le numéro de carte, la date d'expiration et d'autres informations entre les marqueurs ;, = et ?.

On obtient comme résultat :

```
1????0010391000000102904 2=4082657793979904;00000000>26420;420940800000089<08111?800
000000000
```

On remarque que le seul nombre qui est bien balisé (respecte le format de la track 2) et qui contient le nombre exact de chiffres d'une carte bancaire est 4082 6577 9397 9904 (situé entre un « = » et un « ; » et constitué de 16 chiffres). Or en utilisant le site credit-card validator, on se rend compte que cette carte n'existe pas. On retourne notre chaîne de numéros pour retrouver le bon numéro, ce qui est souvent le cas pour les appareils de skimming. En inversant, on obtient ce numéro : 4099 7939 7756 2804. Désormais, on peut lire les renseignements de la carte sur credit-card validator :

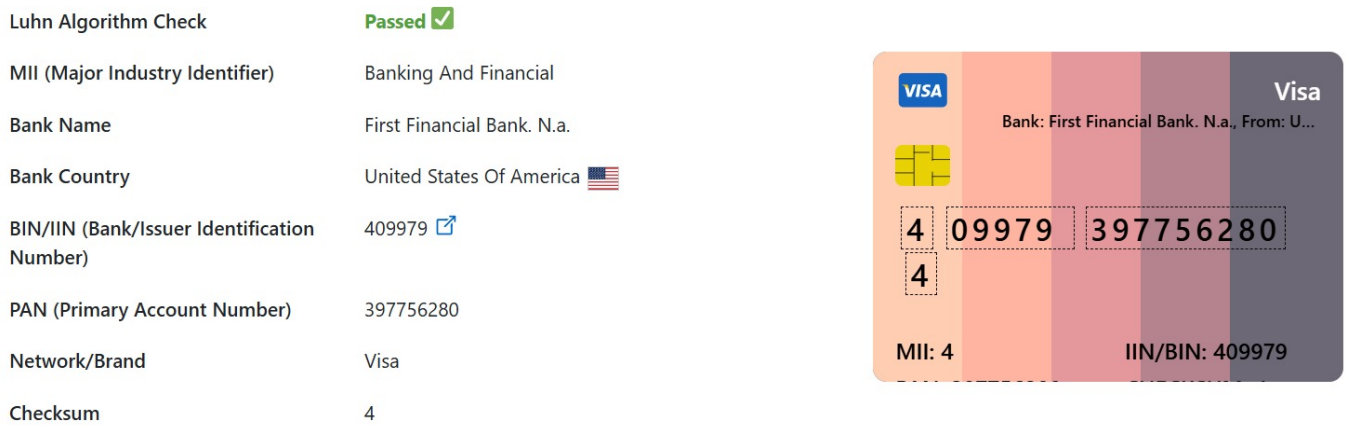


Figure 2.24: Credit Card validator

Partie 3

Analysis

3.1 Identité et lien avec HYDRA

Dans les contacts mail de l'incriminé, on retrouve `therealredskull41@gmail.com`. Red Skull est connu comme une figure majeure d'HYDRA.

Dans les photographies, nous avons retrouvé un logo d'HYDRA, en figure 2.6, qui a été prise à l'Université de Lausanne, le 8 avril 2021.

On peut en déduire que Werner a potentiellement rejoint l'organisation en étant approché à l'Université, qu'il y ait été étudiant ou professeur.

Les messages décodés (voir 2.4.2) vont aussi en ce sens : le *Heil Hydra* 🐙*!*, du 8 avril 2021, semble avoir été envoyé lorsqu'il a rejoint l'organisation.
Ce message illustre son allégeance à HYDRA.

Par ailleurs, le fait que les messages ait été échangés via OmniNotes montre la volonté de dissimuler sa relation avec HYDRA. De plus, dans l'échange du 15 avril, son interlocuteur lui demande de supprimer le message un fois lu.

3.2 Voyage à Genève

La conversation du 15 avril 2021 montre qu'il se rendait à Genève pour rencontrer l'organisation, près du Consulat de Turquie.

On voit sur la figure 2.12 qu'il se trouvait près d'ICC le 15 avril 2021.

On remarque qu'ICC est situé à proximité du consulat de Turquie.



Figure 3.1: Localisation du Consulat de Turquie à Genève

Cela fait donc sens avec les messages : Werner s'est rendu à Genève pour rencontrer HYDRA, peut-être dans un bureau d'ICC, comme on a remarqué précédemment (voir 2.3.2).

3.3 Implication dans le Skimming

Notre suspect se trouvait en possession d'un enregistrement audio cachant des informations bancaires, sans doute volées et ceci constitue un délit.

L'examen des données a mis en évidence plusieurs éléments reliant Werner von Strucker à des activités de skimming :

- Transactions en Bitcoin Cash : les adresses BCH retrouvées (figure 2.14) révèlent un schéma de transferts multiples entre différentes adresses créées et utilisées le 18 avril 2021. Ce mode d'opération correspond à une tentative claire d'obfuscation visant à masquer la provenance et la destination réelle des fonds. Ce type de fragmentation et de réinjection est fréquemment observé dans les réseaux de fraude bancaire.
- skimming : le fichier `2021_04_09T1621` contient une piste correspondant aux enregistrements typiques d'un audioskim. L'analyse avec Magstripper puis le décodage manuel de la Track 2 ont permis d'identifier un numéro de carte valide (4099 7939 7756 2804). Cela confirme que l'appareil détenu par Werner contenait bien des données issues d'un dispositif de skimming installé sur un ATM.

  De plus, d'après les échanges du 18 avril 2021, Werner aurait créer des problèmes à un Postomat, ce qui renvoie aux méthodes de skimming précédemment expliquée.
- Corrélation temporelle : la date de l'enregistrement (9 avril 2021) coïncide avec la découverte d'un skimmer sur un distributeur de la Poste suisse à Lausanne (voir Identification). Cette concordance renforce la responsabilité du suspect.
- Tentative d'utilisation : enfin, Werner a été arrêté avec un billet d'avion Genève - New York acheté via une carte bancaire volée. Cet élément complète la chaîne probatoire reliant la possession des données frauduleuses à leur usage concret.

En conclusion, les preuves matérielles (transactions crypto, audio de piste magnétique, billet d'avion) établissent l'implication directe de Werner von Strucker dans un réseau organisé de fraude bancaire par skimming, en lien avec HYDRA.

Conclusion

Synthèse de l'enquête

Suite à l'analyse forensique du Samsung Galaxy S10 saisi sur Werner von Strucker, cette investigation a permis d'établir de manière irréfutable plusieurs éléments probants :

Appartenance à l'organisation HYDRA

- **Identité confirmée** : Werner von Strucker, personnage fictif lié à l'univers Marvel/HYDRA
- **Contacts criminels** : Correspondance avec `therealredskull41@gmail.com` (Red Skull)
- **Preuves visuelles** : Photo du logo HYDRA prise à l'Université de Lausanne le 8 avril 2021
- **Communications chiffrées** : Messages décodés via OmniNotes révélant *Heil Hydra* 🐙*!*

Implication dans les activités de skimming

- **Transactions financières** : Réseau d'adresses Bitcoin Cash pour l'obfuscation des fonds
- **Déplacements suspects** : Voyage coordonné à Genève le 15 avril 2021 pour rencontrer l'organisation
- **Tentative de fuite** : Billet d'avion Genève - New York du 20 avril 2021, utilisant une carte bancaire volée

Géolocalisation et timeline

- **Zone d'activité** : Région lausannoise (Bussigny, Prilly, Renens)
- **Point de contact** : International Conference Center (ICC) Genève, près du consulat turc
- **Chronologie établie** : Du 8 avril (recrutement HYDRA) au 20 avril 2021 (arrestation)

Intégrité des preuves

L'intégrité de l'image forensique a été vérifiée par contrôle de l'empreinte SHA2-256 :

`54877505f1b4eb26c4cb6b43fd6338424660c207e678b773044a4a79d6e374b7`

Cette vérification garantit que les données analysées n'ont subi aucune altération depuis leur acquisition le 21 avril 2021, respectant ainsi la chaîne de custody requise pour la recevabilité juridique.

Recevabilité juridique

Cette investigation respecte les standards forensiques internationaux :

- **Acquisition légale** : Saisie effectuée dans le cadre d'une perquisition judiciaire
- **Méthodologie certifiée** : Utilisation d'outils forensiques reconnus (Autopsy)
- **Documentation complète** : Traçabilité de chaque étape d'analyse
- **Principe de Locard** : Préservation de l'intégrité des preuves numériques

Le rapport est donc recevable devant un tribunal selon les standards de la digital forensics et les exigences de l'article 259 du Code de procédure pénale suisse concernant l'administration des preuves.

Évaluation critique

Apports pédagogiques

Ce projet nous a permis de maîtriser :

- La méthodologie forensique en 5 phases (Identification, Collection, Examination, Analysis, Presentation)
- L'utilisation d'Autopsy pour l'analyse d'images mobiles Android
- Le reverse engineering d'applications (déchiffrement OmniNotes)
- L'investigation de fraudes bancaires par skimming et analyse d'enregistrements audio
- La consultation et traçage de transactions crypto-monnaies (Bitcoin Cash)
- La corrélation de preuves multiples (géolocalisation, communications, transactions)

Défis techniques rencontrés

- **Performance** : Temps d'analyse élevé d'Autopsy sur l'image complète (plusieurs heures)
- **Cryptographie** : Déchiffrement manuel des messages AES/ECB nécessitant l'analyse du code source
- **Corrélation** : Mise en relation de preuves dispersées (photos, messages, blockchain, géolocalisation)

Cette investigation démontre l'efficacité des techniques de mobile forensics pour reconstituer des activités criminelles complexes, même face à des tentatives de dissimulation par chiffrement.

Sources

- How Magnetic Stripe Readers Work - https://www.youtube.com/watch?v=71_8NHHD8OI
- Skimming - https://www.fbi.gov/how-we-can-help-you/scams-and-safety/common-frauds-and-scams/skimming
- Magstripe Decoder `README` - https://github.com/jtarrio/MagstripeDecoder